Construction of BST-For Given Elements

```
class BST {

    // Node class
    static class Node {
        int key;
        Node left, right;

        Node(int item) {
            key = item;
            left = right = null;
        }
    }

    Node root;

    // Insert method (constructs BST)
    Node insert(Node root, int key) {

        // If tree is empty → create node
        if (root == null) {
            return new Node(key);
        }

        // Go left
        if (key < root.key) {
            root.left = insert(root.left, key);
        }
        // Go right
        else if (key > root.key) {
            root.right = insert(root.right, key);
        }

        return root;
    }

    public static void main(String[] args) {

        BST tree = new BST();

        // Construct BST
        tree.root = tree.insert(tree.root, 50);
        tree.insert(tree.root, 30);
        tree.insert(tree.root, 70);
        tree.insert(tree.root, 20);
        tree.insert(tree.root, 40);

        System.out.println("BST constructed successfully.");
    }
}
```

Search an Element in BST

```
class BST {

    static class Node {
        int key;
        Node left, right;

        Node(int item) {
            key = item;
            left = right = null;
        }
    }

    Node root;

    // INSERT (for building tree)
    Node insert(Node root, int key) {
        if (root == null)
            return new Node(key);

        if (key < root.key)
            root.left = insert(root.left, key);
        else if (key > root.key)
            root.right = insert(root.right, key);

        return root;
    }

    // SEARCH
    boolean search(Node root, int key) {

        // Base case
        if (root == null)
            return false;

        if (root.key == key)
            return true;

        if (key < root.key)
            return search(root.left, key);

        return search(root.right, key);
    }

    public static void main(String[] args) {

        BST tree = new BST();

        tree.root = tree.insert(tree.root, 50);
        tree.insert(tree.root, 30);
        tree.insert(tree.root, 70);
        tree.insert(tree.root, 20);
        tree.insert(tree.root, 40);
```

```
        int key = 40;

        if (tree.search(tree.root, key))
            System.out.println("Element found");
        else
            System.out.println("Element not found");
    }
}
```

Construction of BST- With In Order Traversal

```
class BST {

    // Step 1: Node class
    static class Node {
        int key;
        Node left, right;

        Node(int item) {
            key = item;
            left = right = null;
        }
    }

    Node root;

    // Step 2: Insert method
    Node insert(Node root, int key) {

        if (root == null) {
            return new Node(key);
        }

        if (key < root.key) {
            root.left = insert(root.left, key);
        } else if (key > root.key) {
            root.right = insert(root.right, key);
        }

        return root;
    }

    // Step 3: Inorder traversal
    void inorder(Node root) {
        if (root != null) {
            inorder(root.left);
            System.out.print(root.key + " ");
            inorder(root.right);
        }
    }

    // Step 4: Main method
```

```
    public static void main(String[] args) {

        BST tree = new BST();

        tree.root = tree.insert(tree.root, 50);
        tree.insert(tree.root, 30);
        tree.insert(tree.root, 70);
        tree.insert(tree.root, 20);
        tree.insert(tree.root, 40);

        System.out.println("BST Inorder Traversal:");
        tree.inorder(tree.root);
    }
}
```

code snippets for Preorder and postorder

```
void preorder(Node root) {
if (root != null) {
System.out.print(root.key + " "); // Root
preorder(root.left); // Left
preorder(root.right); // Right
}
}

void postorder(Node root) {
    if (root != null) {
        postorder(root.left);             // Left
        postorder(root.right);            // Right
        System.out.print(root.key + " "); // Root
    }
}
```

BST (Insertion+ Deletion of Node)

```
class BST {

    // Node class
    static class Node {
        int key;
        Node left, right;

        Node(int item) {
            key = item;
            left = right = null;
        }
    }

    Node root;

    // INSERT
    Node insert(Node root, int key) {
        if (root == null) return new Node(key);
```

```
    if (key < root.key)
        root.left = insert(root.left, key);
    else if (key > root.key)
        root.right = insert(root.right, key);

    return root;
}

// FIND MIN (used in deletion)
Node minValueNode(Node root) {
    Node current = root;
    while (current.left != null)
        current = current.left;
    return current;
}

// DELETE
Node delete(Node root, int key) {

    // Step 1: Find node
    if (root == null) return root;

    if (key < root.key)
        root.left = delete(root.left, key);

    else if (key > root.key)
        root.right = delete(root.right, key);

    else {
        // Node found

        // Case 1 & 2: One or no child
        if (root.left == null)
            return root.right;

        else if (root.right == null)
            return root.left;

        // Case 3: Two children
        Node temp = minValueNode(root.right);
        root.key = temp.key;
        root.right = delete(root.right, temp.key);
    }

    return root;
}

public static void main(String[] args) {

    BST tree = new BST();

    tree.root = tree.insert(tree.root, 50);
    tree.insert(tree.root, 30);
    tree.insert(tree.root, 70);
```

```
        tree.insert(tree.root, 20);
        tree.insert(tree.root, 40);
        tree.insert(tree.root, 60);
        tree.insert(tree.root, 80);

        // Delete element
        tree.root = tree.delete(tree.root, 20);

        System.out.println("Deletion completed.");
    }
}
```

Construction of BST-With Tree Structure

```
class BST {

// Node class
static class Node {
int key;
Node left, right;

Node(int item) {
key = item;
left = right = null;
}
}

Node root;

// Insert method
Node insert(Node root, int key) {
if (root == null) {
return new Node(key);
}

if (key < root.key) {
root.left = insert(root.left, key);
} else if (key > root.key) {
root.right = insert(root.right, key);
}

return root;
}

// Print BST as tree (sideways)
void printTree(Node root, int space) {
if (root == null)
return;

space += 5;
```

```
// Print right subtree first
printTree(root.right, space);

// Print current node
System.out.println();
for (int i = 5; i < space; i++)
System.out.print(" ");
System.out.println(root.key);

// Print left subtree
printTree(root.left, space);
}

public static void main(String[] args) {

BST tree = new BST();

tree.root = tree.insert(tree.root, 50);
tree.insert(tree.root, 30);
tree.insert(tree.root, 70);
tree.insert(tree.root, 20);
tree.insert(tree.root, 40);

System.out.println("BST Structure:\n");
tree.printTree(tree.root, 0);
}
}
```